🚀 Lecture 04.02: String Slicing Mechanics & Substring Extraction

📌 Executive Architectural Summary

String slicing allows extracting contiguous or strided sub-sequences using the extended slice syntax `S[start:stop:step]`. Under the hood, Python constructs a `slice(start, stop, step)` object passed to `__getitem__()`.

🔬 Core Engineering Mechanics

1. Extended Slice Formalism

For an expression $S[i : j : k]$:

- i (`start`): Inclusive lower bound (defaults to 0 if $k > 0$, or -1 if $k < 0$).
- j (`stop`): Exclusive upper bound (defaults to N if $k > 0$, or $-\infty$ if $k < 0$).
- k (`step`): Stride increment; defines the sampling frequency and traversal direction.

2. Negative Step & String Reversal

When $k = -1$:

> $$S[:: -1] = S[N - 1 : -1 : -1]$$

This traverses the string backwards from the final code point to the initial code point, enabling clean $O(N)$ string reversal.

💻 Production Python Implementation

```
text = "ArtificialIntelligence"

print("Original Text:     ", text)
print("Slice [0:10]:      ", text[0:10])      # 'Artificial'
print("Slice [10:]:       ", text[10:])       # 'Intelligence'
print("Step slice [::2]:  ", text[::2])       # Every second character
```

```
print("Reversed [::-1]:   ", text[::-1])      # Full string reversed

# Palindrome Verification via Slicing
word = "racecar"
is_palindrome = (word == word[::-1])
print(f"Is '{word}' a palindrome? {is_palindrome}")
```